

CSC 308: Operating Systems

Week 1: Fundamentals of OS Design and Implementation

1. What is an Operating System?

An Operating System (OS) is the core system software that acts as an intermediary between the computer hardware and the user/applications. A common academic definition is that the OS is "the one program running at all times on the computer" (usually called the kernel), with all else being system or application programs.

To fully understand an OS, we must look at it from two distinct viewpoints:

The User View

From the user's perspective, the primary goal of the OS is **convenience** and **ease of use**, often prioritizing user experience over raw performance.

- **Abstraction:** It abstracts away the complex, low-level details of the hardware. Users don't need to know how to move the disk head to sector 42 to save a document; they just click "Save."
- **Interfaces:** It provides logical interfaces for the user to interact with the machine, ranging from Graphical User Interfaces (GUIs) with windows and icons, to Command-Line Interfaces (CLIs), and even voice or touch inputs in modern devices.
- **Note on Embedded Systems:** In embedded systems (like microwaves or industrial sensors), the "user view" is minimal or non-existent, and the OS is designed almost entirely to run without user intervention.

The System View

From the computer's perspective, the OS is a **Resource Allocator** and a **Control Program**.

- **Resource Allocator:** A computer has limited CPU time, memory space, file storage, and I/O bandwidth. The OS acts as the manager of these resources. When multiple programs run simultaneously, the OS allocates these resources fairly and efficiently, resolving conflicting requests (e.g., two programs trying to print at the same time).
- **Control Program:** It oversees the execution of user programs to prevent errors and improper use of the computer. It ensures that a bug in a user application (like an infinite loop or an attempt to access restricted memory) does not crash the entire system.

System Architecture Hierarchy

A standard computer system can be divided into four conceptual layers, building from the physical components up to the end-user experience:

1. **Hardware (The Machine Level):** Provides the fundamental physical computing resources.
 - *Central Processing Unit (CPU):* The brain of the computer, containing the Arithmetic Logic Unit (ALU), Control Unit, and high-speed registers used for executing instructions.
 - *Memory Hierarchy:* Ranging from extremely fast but volatile CPU caches (L1/L2/L3) and Main Memory (RAM), down to slower, high-capacity, non-volatile secondary storage (Magnetic HDDs, Solid State Drives).
 - *I/O Devices & Buses:* The physical interfaces (keyboards, monitors, network interface cards) and the communication pathways (buses) that connect them to the CPU and memory.
2. **Operating System (The Kernel Level):** The highly privileged core software that controls and coordinates the use of the hardware.
 - *Resource Management:* It dictates exactly which application gets access to the CPU, memory, and I/O devices, and limits for how long they can use them.
 - *Hardware Abstraction:* It hides the complexity and diversity of the underlying hardware through the use of device drivers, presenting a uniform, standard interface to the software layers above.
3. **System and Application Programs (The User Space):** The software layer where actual user tasks are executed, running in a restricted, unprivileged mode (User Mode).
 - *System Programs (Utilities):* Software provided with the OS that is *not* part of the core kernel. This includes command-line shells, background services (daemons), compilers, text editors, and file manipulation utilities. They provide a convenient environment for program development and system administration.
 - *Application Programs:* Third-party or user-installed software designed to solve specific computing problems, perform business logic, or provide entertainment (e.g., web browsers, word processors, video games, complex database management systems).
4. **Users (The Entity Level):** The agents actively interacting with the computer system to solve problems.
 - *Human Users:* Interacting directly via GUIs, CLIs, or touch interfaces.
 - *Machine Users:* Other computers, embedded devices, or automated scripts interacting with the system over a network or through programmatic Application Programming Interfaces (APIs).

2. History and Evolution of Operating Systems

The progression of OS design has historically been driven by the constant need to maximize

hardware utilization and operational efficiency as hardware costs decreased and processing power increased.

Phase 1: Bare Machine / Serial Processing (1940s - 1950s)

- **Concept:** No operating system existed. Programmers interacted directly with the hardware using physical plugboards, punch cards, or paper tape.
- **Characteristics:** The programmer was also the operator.
- **Drawback:** Immense wasted time during the setup and teardown of jobs. If a programmer booked an hour of computer time and finished in 15 minutes, the machine sat idle for 45 minutes.

Phase 2: Simple Batch Systems (1950s - 1960s)

- **Concept:** Similar jobs were grouped (batched) together by a human operator onto magnetic tape to reduce setup time.
- **The Resident Monitor:** The first primitive OS. It automatically transferred control from one job to the next.
- **Spooling (Simultaneous Peripheral Operations On-Line):** Introduced to use disk storage as a buffer. Cards were read directly to disk, and the CPU pulled jobs from the disk. This allowed the CPU to overlap its computation with the I/O of other jobs.
- **Drawback:** Despite spooling, the CPU was still frequently idle. Mechanical I/O devices are orders of magnitude slower than electronic CPUs. When a job needed to read data, the CPU stopped and waited.

Phase 3: Multiprogramming (Late 1960s - The Major Leap)

- **Concept:** The OS keeps several jobs in memory simultaneously (the "job pool"). The OS picks and begins to execute one of the jobs in memory.
- **The Breakthrough:** When a job has to wait for an I/O operation to complete, the OS does *not* sit idle. Instead, it saves the state of the current job and switches the CPU to another job.
- **Result:** Drastically increased CPU utilization. The CPU is almost never idle as long as there is at least one job ready to execute. This era required new OS features like memory management (to protect jobs from each other) and CPU scheduling algorithms.

Phase 4: Time-Sharing / Multitasking Systems (1970s+)

- **Concept:** A logical extension of multiprogramming designed for interactive use. The CPU

executes multiple jobs by switching among them very rapidly (e.g., every 10-100 milliseconds).

- **Result:** Brought about interactive computing. Because the switches occur so quickly, users feel they each have their own dedicated computer, allowing multiple users to share a single mainframe or server simultaneously (e.g., the development of UNIX).
- **New Challenges:** This necessitated complex file systems, strict memory protection protocols, and advanced virtual memory management (to handle programs larger than physical RAM).

Phase 5: Personal Computing & Beyond (1980s - Present)

- **Concept:** Hardware became cheap enough that single users could own dedicated machines. Early PC OSs (like MS-DOS) sacrificed multiprogramming for simplicity, but eventually adopted time-sharing concepts (Windows, macOS, Linux). This era introduced robust Graphical User Interfaces (GUIs) and built-in networking protocols.

3. Types of Operating Systems

Depending on the environment, hardware architecture, and use case, OS architectures are tailored for specific requirements:

- **Real-Time OS (RTOS):** Used when strict, rigid time constraints are placed on the operation of a processor or the flow of data. Examples include VxWorks and QNX.
 - *Hard Real-Time:* Missing a deadline is considered a total system failure (e.g., pacemakers, anti-lock brakes, weapon systems). Processing must be guaranteed within specific microsecond margins.
 - *Soft Real-Time:* Missing a deadline degrades performance but is tolerable (e.g., live video streaming, virtual reality, audio processing).
- **Multiprocessor Systems (Parallel Systems):** Systems with two or more CPUs in close communication, sharing the computer bus, clock, and memory.
 - *Symmetric Multiprocessing (SMP):* Each processor runs an identical copy of the OS and they share the workload equally. Most modern standard OSs (Windows, Linux, macOS) are SMP.
 - *Asymmetric Multiprocessing:* Each processor is assigned a specific task by a master processor.
- **Clustered Systems:** Similar to multiprocessor systems, but composed of multiple individual, distinct computer systems joined together via a high-speed network (like a Storage Area Network or Infiniband). Used heavily for High-Availability (if one node fails, another takes over) and High-Performance Computing (HPC).
- **Distributed OS:** Manages a group of physically separate computers connected via a network, masking the network entirely and making them appear as a single, coherent local system to the user.

- **Network OS:** Runs on a dedicated server and provides the capability to manage data, users, groups, security, and networking functions across a Local Area Network (LAN). The users are aware of the network.
- **Mobile OS:** Designed specifically for the resource constraints (battery life, limited memory, thermal throttling) and interface paradigms (touch screens, GPS, accelerometers) of smartphones and tablets (e.g., Android, iOS).

4. Fundamental OS Operations

To maintain strict control over the hardware, protect data integrity, and manage user applications, the OS relies on several specific hardware-level mechanisms:

Interrupt-Driven Execution

Modern operating systems are inherently interrupt-driven. An OS will start up, initialize its state, start system services, and then wait for an event to occur.

- **Hardware Interrupts:** Signals sent to the CPU by external hardware devices (e.g., a keyboard press, a mouse click, a network packet arriving, or a disk drive completing a data read).
- **Software Interrupts (Traps/Exceptions):** Signals generated by software either due to an error (e.g., division by zero, invalid memory access) or a deliberate request for an OS service (a system call).

When an interrupt occurs, the hardware automatically suspends the currently executing program, saves its state (registers and program counter), and transfers control to the OS's **Interrupt Service Routine (ISR)** through an array of memory addresses called the Interrupt Vector Table. Once the OS handles the event, it restores the interrupted program's state.

Dual-Mode Operation

To protect the OS from errant or malicious user programs, the CPU hardware provides two distinct modes of execution, tracked by a hardware flag called the "Mode Bit":

1. **User Mode (Mode bit = 1):** The system is executing standard application code. In this mode, the CPU prevents the execution of **Privileged Instructions**.
2. **Kernel Mode / Supervisor Mode (Mode bit = 0):** The system is executing OS code. All hardware instructions, memory spaces, and I/O devices are directly accessible.

Examples of Privileged Instructions (Only allowed in Kernel Mode):

- Disabling or enabling interrupts.
- Changing the Mode Bit from User to Kernel.
- Accessing hardware I/O directly.
- Modifying memory management registers (like page tables).

The System Call Interface

When a user application needs to perform a privileged action (such as allocating memory, writing a file to disk, or opening a network connection), it cannot do so directly. It must trigger a **System Call**.

1. The application executes a special software trap instruction.
2. The hardware switches the Mode Bit from 1 to 0 (Kernel Mode).
3. Control transfers to the OS kernel.
4. The OS verifies the parameters of the request to ensure security and validity.
5. The OS executes the requested privileged action.
6. The OS switches the Mode Bit back to 1 (User Mode) and returns control (and any data) to the application.

The Hardware Timer

To ensure that the OS maintains control over the CPU, the system utilizes a hardware timer.

- **The Problem:** What if a user program enters an infinite loop, or maliciously refuses to yield the CPU? The OS would be locked out forever because the OS is just software; it needs the CPU to execute its own instructions.
- **The Solution:** Before transferring control to a user program, the OS sets a hardware timer to interrupt after a specific period (e.g., 10 milliseconds). When the timer counts down to zero, it generates a hardware interrupt, forcing the CPU to switch back to Kernel Mode and return control to the OS, allowing it to schedule another process.